

# Jobsheet Praktikum: Pertemuan 4

## Relasi Kelas: Aggregation, Composition, dan Dependency

---

|             |                                                                                                                                                                       |
|-------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Mata Kuliah | Praktikum Pemrograman Berbasis Objek (RTI253008)                                                                                                                      |
| Pertemuan   | 4 (Minggu 4)                                                                                                                                                          |
| Durasi      | 1 × 4 × 50' praktikum; 1 × 1 × 50' tugas/laporan mandiri                                                                                                              |
| Sub-CPMK    | Mengimplementasikan relasi antar kelas (has-a) → agregasi, komposisi, dependensi (uses-a), dan multiplicity, dalam kode Java                                          |
| Kode Awal   | tidak ada prasyarat kode wajib; tiap percobaan dimulai dari package baru yang berdiri sendiri (bisa lanjutan proyek dari jobsheet sebelumnya)                         |
| Kode Akhir  | proyek berisi 6 package <code>relasiclass.percobaan1</code> s.d. <code>percobaan6</code> , masing-masing berisi implementasi Aggregation, Composition, dan Dependency |

---

### A. Capaian Praktikum

Setelah menyelesaikan jobsheet ini, mahasiswa mampu:

1. Menjelaskan perbedaan **Association**, **Aggregation**, **Composition**, dan **Dependency** (uses-a)
2. Mengimplementasikan relasi **Aggregation** antar kelas untuk kasus satu-ke-satu, relasi ganda (satu kelas berelasi dengan lebih dari satu kelas lain), dan relasi dengan beberapa role ke kelas yang sama.
3. Mengimplementasikan relasi **Composition**, termasuk pola satu-ke-banyak menggunakan array of object, dan memahami mengapa “tidak ada setter” adalah bagian penting dari desainnya.
4. Mengimplementasikan relasi **Dependency** (uses-a) melalui parameter method, dan membedakannya dari Aggregation.
5. Menerjemahkan diagram kelas UML, termasuk notasi diamond (◇/◆), garis putus-putus, dan multiplicity, menjadi kode Java yang benar dan konsisten dengan diagramnya.

### B. Persiapan dan Prasyarat

- **Alat:** JDK 17 atau lebih baru, VSCode (editor yang kita pakai sepanjang praktikum ini).
- **Prasyarat:** sudah menyelesaikan jobsheet kelas dan objek (mendefinisikan kelas, atribut, konstruktor, method, serta memahami bahwa variabel objek adalah **referensi**, bukan objeknya sendiri, ini akan sering dipakai di jobsheet ini).
- **Proyek:** setiap percobaan memakai package baru bernama `<identifier>.relasiclass.percobaanN` (ganti `<identifier>` dengan identitas mahasiswa, contoh: `ac.id.polinema`). Package terpisah untuk tiap percobaan supaya nama kelas yang sama (mis. `Mobil` pada Percobaan 2 dan 5) tidak saling bentrok.

 **Info:** Konvensi penamaan package: nama domain institusi dibalik urutannya, “polinema.ac.id” menjadi `ac.id.polinema`, lalu ditambah `.relasiclass.percobaanN`.

**Peta relasi yang akan dipraktikkan**, supaya mahasiswa tahu ke mana arah jobsheet ini, berikut ringkasannya sebelum mulai:

| Percobaan | Studi Kasus                 | Relasi                    | Konsep Baru                                       |
|-----------|-----------------------------|---------------------------|---------------------------------------------------|
| 1         | Laptop – Processor          | Aggregation 1-1           | constructor & setter injection                    |
| 2         | Pelanggan – Mobil – Sopir   | Aggregation ganda         | 1 kelas berelasi ke >1 kelas lain                 |
| 3         | KeretaApi – Pegawai         | Aggregation, 2 role       | role name, <b>null</b> , NullPointerException     |
| 4         | Gerbang – Kursi – Penumpang | Composition + Aggregation | array of object, multiplicity, guard clause       |
| 5         | Mobil – Mesin               | Composition               | objek dibuat sendiri di constructor, tanpa setter |
| 6         | Laptop – Printer            | Dependency (uses-a)       | objek hanya lewat parameter, tidak disimpan       |

## C. Langkah Kerja

Setiap Percobaan di bawah ini berdiri sendiri (package terpisah) dan dipecah menjadi langkah-langkah kecil. Kerjakan berurutan, tiap langkah dibangun di atas langkah sebelumnya di Percobaan yang sama.

### Percobaan 1: Aggregation Satu-ke-Satu (Laptop dan Processor)

Sebuah **Laptop** memiliki **Processor**. Diperinci lebih jauh, Processor punya atributnya sendiri (**merk**, **cache**), artinya **Processor** adalah objek dari kelas tersendiri, bukan sekadar String atau angka. Karena atribut **proc** pada Laptop bertipe objek **Processor**, kelas Laptop dikatakan **memiliki relasi (has-a)** dengan kelas Processor.

#### Langkah 1: Diagram kelas

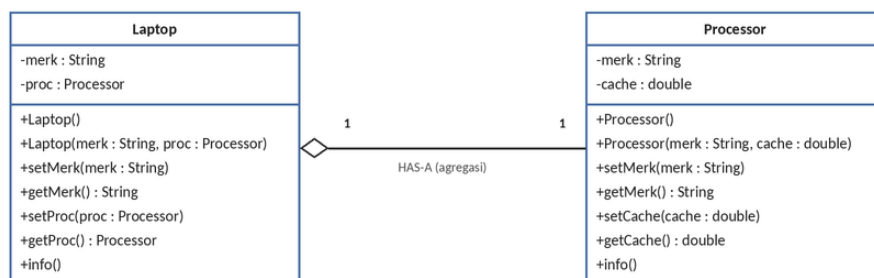


Figure 1: Diagram kelas Laptop dan Processor, diamond ◇ di sisi Laptop menandakan Laptop adalah “whole”, Processor adalah “part”. Multiplicity 1-1: satu Laptop tepat punya satu Processor.

**Catatan Konsep:** Diamond ◇ (kosong) = **Aggregation**: part (Processor) tetap bisa berdiri sendiri walau whole-nya (Laptop) dihapus. Kita akan buktikan lewat kode: perhatikan baris mana yang membuat objek Processor. Di dalam class Laptop, atau di luar?

#### Langkah 2: Buat class Processor

Buat package `id.ac.polinema.relasiclass.percobaan1`, lalu buat class **Processor** dengan dua atribut private berikut:

```

1 package id.ac.polinema.relasiclass.percobaan1;
2
3 public class Processor {
4     private String merk;
5     private double cache;
6 }

```

*Kode 1: Processor.java: atribut merk dan cache*

### Langkah 3: Tambah constructor Processor

Tambahkan constructor default dan constructor berparameter:

```

1 package id.ac.polinema.relasiclass.percobaan1;
2
3 public class Processor {
4     private String merk;
5     private double cache;
6
7     public Processor() {
8     }
9
10    public Processor(String merk, double cache) {
11        this.merk = merk;
12        this.cache = cache;
13    }
14 }

```

*Kode 2: Processor.java dengan dua constructor*

### Langkah 4: Tambah setter, getter, dan info()

```

1 package id.ac.polinema.relasiclass.percobaan1;
2
3 public class Processor {
4     private String merk;
5     private double cache;
6
7     public Processor() {
8     }
9
10    public Processor(String merk, double cache) {
11        this.merk = merk;
12        this.cache = cache;
13    }
14
15    public void setMerk(String merk) {
16        this.merk = merk;
17    }
18
19    public String getMerk() {
20        return merk;
21    }
22
23    public void setCache(double cache) {
24        this.cache = cache;
25    }
26
27    public double getCache() {
28        return cache;
29    }
30
31    public void info() {
32        System.out.printf("Merk Processor = %s\n", merk);
33        System.out.printf("Cache Memory = %.2f\n", cache);
34    }
35 }

```

*Kode 3: Processor.java lengkap dengan setter, getter, dan info()*

⚠ **Jika gagal:** kalau VSCode menggarisbawahi merah pada `%s/%.2f`, cek dulu tanda kutip dan titik-koma, `printf` sangat sensitif terhadap format string yang tidak lengkap.

## Langkah 5: Buat class Laptop

Sekarang buat class **Laptop** pada package yang sama. Perhatikan atribut keduanya:

```
1 package id.ac.polinema.relaiclass.percobaan1;
2
3 public class Laptop {
4     private String merk;
5     private Processor proc;
6 }
```

*Kode 4: Laptop.java: atribut merk (String) dan proc (Processor)*

📌 **Catatan Konsep:** `merk` bertipe `String`, tapi `proc` bertipe `Processor`, sebuah **objek**, bukan tipe primitif. Baris `private Processor proc;` inilah yang secara kode menyatakan “Laptop punya (has-a) Processor”.

## Langkah 6: Tambah constructor, setter, dan getter Laptop

```
1 package id.ac.polinema.relaiclass.percobaan1;
2
3 public class Laptop {
4     private String merk;
5     private Processor proc;
6
7     public Laptop() {
8     }
9
10    public Laptop(String merk, Processor proc) {
11        this.merk = merk;
12        this.proc = proc;
13    }
14
15    public void setMerk(String merk) {
16        this.merk = merk;
17    }
18
19    public String getMerk() {
20        return merk;
21    }
22
23    public void setProc(Processor proc) {
24        this.proc = proc;
25    }
26
27    public Processor getProc() {
28        return proc;
29    }
30 }
```

*Kode 5: Laptop.java dengan constructor, setter, dan getter*

## Langkah 7: Tambah method info() dengan delegasi

```
1 public void info() {
2     System.out.println("Merk Laptop = " + merk);
3     proc.info();
4 }
```

*Kode 6: Tambahan method info() pada Laptop.java (disisipkan sebelum kurung kurawal penutup class)*

**Catatan Konsep:** `proc.info()` adalah **delegasi**: Laptop tidak mencetak detail Processor sendiri, melainkan “menyuruh” objek Processor yang dimilikinya untuk mencetak dirinya sendiri. Ini pola umum pada relasi has-a: whole memanggil method milik part-nya.

## Langkah 8: Uji dengan constructor berparameter

Buat class `MainPercobaan1` berisi method `main()`:

```
1 package id.ac.polinema.relasiclass.percobaan1;
2
3 public class MainPercobaan1 {
4     public static void main(String[] args) {
5         Processor p = new Processor("Intel i5", 3);
6         Laptop l = new Laptop("Thinkpad", p);
7         l.info();
8     }
9 }
```

*Kode 7: MainPercobaan1.java: membuat Processor lalu meng-inject-nya ke Laptop*

☒ **Checkpoint:** output menampilkan:

Merk Laptop = Thinkpad lalu Merk Processor = Intel i5 dan Cache Memory = 3.00.

## Langkah 9: Uji dengan constructor default dan setter injection

Tambahkan baris berikut ke akhir `main()` (jangan hapus baris sebelumnya):

```
1 Processor p1 = new Processor();
2 p1.setMerk("Intel i5");
3 p1.setCache(4);
4 Laptop l1 = new Laptop();
5 l1.setMerk("Thinkpad");
6 l1.setProc(p1);
7 l1.info();
```

*Kode 8: Tambahan pada MainPercobaan1.java: membuat objek lewat constructor default lalu diisi lewat setter*

☒ **Checkpoint:** muncul baris output kedua, isinya sama persis dengan baris pertama tapi cache-nya 4.00.

## Langkah 10: Objek Processor dibuat langsung sebagai argumen (anonymous object)

Tambahkan satu baris lagi di akhir `main()`:

```
1 Laptop l2 = new Laptop("Thinkpad",
2     new Processor("Intel i5", 3));
3 l2.info();
```

*Kode 9: Tambahan pada MainPercobaan1.java: objek Processor dibuat inline sebagai argumen*

**Catatan Konsep:** Outputnya identik dengan langkah 8, meski `new Processor(...)` ditulis langsung di dalam argumen. Yang menentukan relasi ini tetap **Aggregation** bukan lokasinya ditulis (di variabel terpisah atau inline), melainkan **class mana yang memanggil `new Processor(...)`**. Di sini yang memanggil adalah kode di `MainPercobaan1`, bukan kode di dalam class `Laptop` sendiri. Bandingkan dengan Percobaan 5 nanti.

## Pertanyaan Percobaan 1

1. Di dalam class Processor dan class Laptop, terdapat method setter dan getter untuk masing-masing atributnya. Apakah gunanya method setter dan getter tersebut?
2. Di dalam class Processor dan class Laptop, masing-masing terdapat konstruktor default dan konstruktor berparameter. Bagaimanakah beda penggunaan dari kedua jenis konstruktor tersebut?
3. Perhatikan class Laptop, di antara 2 atribut yang dimiliki (**merk** dan **proc**), atribut manakah yang bertipe object? Baris kode manakah yang menunjukkan bahwa class Laptop memiliki relasi dengan class Processor?
4. Perhatikan pada class Laptop, apakah guna dari sintaks `proc.info()`?
5. Pada Langkah 8, objek **p** dibuat lebih dulu baru diberikan ke constructor Laptop. Pada Langkah 10, objek Processor dibuat langsung di dalam argumen constructor Laptop (tanpa variabel **p**). Apakah keduanya menghasilkan output yang berbeda? Mengapa?
6. **Secara kode**, apakah relasi Laptop-Processor pada percobaan ini termasuk Aggregation atau Composition? Tunjukkan baris kode yang menjadi bukti jawabanmu.
7. Andaikan constructor Laptop diubah menjadi seperti berikut, sehingga Processor dibuat **sendiri di dalam** Laptop, bukan diterima sebagai parameter:

```
public Laptop (String merk) {  
    this.merk = merk;  
    this.proc = new Processor ("Generic", 1);  
}
```

Apakah relasi Laptop-Processor pada versi ini masih Aggregation? Jelaskan alasannya (jawaban ini akan kita buktikan sendiri lewat kode pada Percobaan 5).

## Percobaan 2: Aggregation dengan Relasi Ganda (Rental Mobil)

Sebuah sistem rental mobil: **Pelanggan** bisa menyewa **Mobil** sekaligus **Sopir**. Biaya sopir dan biaya sewa mobil dihitung per hari. Berbeda dari Percobaan 1, di sini satu kelas (Pelanggan) berelasi dengan **dua** kelas lain sekaligus.

### Langkah 1: Diagram kelas

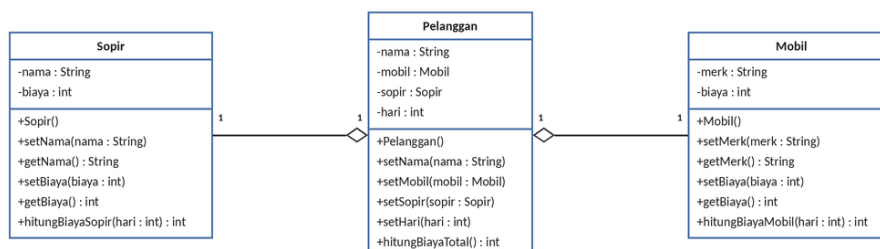


Figure 2: Diagram kelas Sopir, Pelanggan, dan Mobil. Diamond ◇ berada di sisi Pelanggan pada kedua relasi. Pelanggan adalah “whole” untuk Mobil maupun Sopir.

⚠ **Jika gagal:** atribut **Mobil** yang benar adalah **merk**, bukan “nama”. Pastikan diagram dan kode kita konsisten memakai **merk/setMerk/getMerk**, sesuai kode contoh di bawah.

## Langkah 2: Buat class Mobil

Buat package `id.ac.polinema.relasticlass.percobaan2`, lalu buat class `Mobil` dengan constructor default, setter/getter, dan `hitungBiayaMobil`:

```
1 package id.ac.polinema.relasticlass.percobaan2;
2
3 public class Mobil {
4     private String merk;
5     private int biaya;
6
7     public Mobil() {
8     }
9
10    public void setMerk(String merk) {
11        this.merk = merk;
12    }
13
14    public String getMerk() {
15        return merk;
16    }
17
18    public void setBiaya(int biaya) {
19        this.biaya = biaya;
20    }
21
22    public int getBiaya() {
23        return biaya;
24    }
25
26    public int hitungBiayaMobil(int hari) {
27        return biaya * hari;
28    }
29 }
```

*Kode 10: Mobil.java lengkap*

## Langkah 3: Buat class Sopir

```
1 package id.ac.polinema.relasticlass.percobaan2;
2
3 public class Sopir {
4     private String nama;
5     private int biaya;
6
7     public Sopir() {
8     }
9
10    public void setNama(String nama) {
11        this.nama = nama;
12    }
13
14    public String getNama() {
15        return nama;
16    }
17
18    public void setBiaya(int biaya) {
19        this.biaya = biaya;
20    }
21
22    public int getBiaya() {
23        return biaya;
24    }
25
26    public int hitungBiayaSopir(int hari) {
27        return biaya * hari;
28    }
29 }
```

*Kode 11: Sopir.java lengkap (strukturnya sengaja mirip Mobil.java)*

 **Info:** Sopir dan Mobil punya bentuk yang mirip (biaya dihitung per hari), tapi keduanya **kelas yang berbeda**. Pelanggan akan menyimpan referensi ke masing-masing secara terpisah.

#### Langkah 4: Buat class Pelanggan (whole untuk Mobil dan Sopir)

```
1 package id.ac.polinema.relasticlass.percobaan2;
2
3 public class Pelanggan {
4     private String nama;
5     private Mobil mobil;
6     private Sopir sopir;
7     private int hari;
8
9     public Pelanggan() {
10    }
11
12    public void setNama(String nama) {
13        this.nama = nama;
14    }
15
16    public String getNama() {
17        return nama;
18    }
19
20    public void setMobil(Mobil mobil) {
21        this.mobil = mobil;
22    }
23
24    public Mobil getMobil() {
25        return mobil;
26    }
27
28    public void setSopir(Sopir sopir) {
29        this.sopir = sopir;
30    }
31
32    public Sopir getSopir() {
33        return sopir;
34    }
35
36    public void setHari(int hari) {
37        this.hari = hari;
38    }
39
40    public int getHari() {
41        return hari;
42    }
43 }
```

*Kode 12: Pelanggan.java: dua atribut bertipe objek (mobil, sopir)*

 **Catatan Konsep:** Pelanggan memiliki **dua** atribut bertipe objek sekaligus: **mobil** dan **sopir**. Artinya Pelanggan ber-Aggregation dengan dua kelas berbeda secara bersamaan. Ini yang membedakan Percobaan 2 dari Percobaan 1.

#### Langkah 5: Tambah hitungBiayaTotal()

```
1 public int hitungBiayaTotal() {
2     return mobil.hitungBiayaMobil(hari) + sopir.hitungBiayaSopir(hari);
3 }
```

*Kode 13: Tambahan pada Pelanggan.java (sisipkan sebelum kurung kurawal penutup class)*

**Catatan Konsep:** `mobil.hitungBiayaMobil(hari)` memanggil method milik objek `mobil` yang dipegang Pelanggan, lalu hasilnya dijumlahkan dengan hasil `sopir.hitungBiayaSopir(hari)`. Pelanggan sendiri tidak tahu **caranya** menghitung biaya mobil/sopir. Ia hanya menyuruh objek yang tepat untuk menghitungnya sendiri.

## Langkah 6: Uji di MainPercobaan2

```
1 package id.ac.polinema.relasticlass.percobaan2;
2
3 public class MainPercobaan2 {
4     public static void main(String[] args) {
5         Mobil m = new Mobil();
6         m.setMerk("Avanza");
7         m.setBiaya(350000);
8
9         Sopir s = new Sopir();
10        s.setNama("John Doe");
11        s.setBiaya(200000);
12
13        Pelanggan p = new Pelanggan();
14        p.setNama("Jane Doe");
15        p.setMobil(m);
16        p.setSopir(s);
17        p.setHari(2);
18
19        System.out.println("Biaya Total = " + p.hitungBiayaTotal());
20    }
21 }
```

Kode 14: MainPercobaan2.java

✓ **Checkpoint:** output menampilkan `Biaya Total = 1100000` (= (350000 + 200000) x 2).

⚠ **Jika gagal:** kalau muncul `NullPointerException` pada baris `hitungBiayaTotal()`, cek apakah `p.setMobil(m)` dan `p.setSopir(s)` benar-benar dipanggil **sebelum** `p.hitungBiayaTotal()`. Tanpa itu, `mobil/sopir` di dalam Pelanggan masih `null`. Kita akan bahas error ini lebih dalam di Percobaan 3.

## Langkah 7: Navigasi rantai objek (a.b().c())

Tambahkan satu baris lagi di akhir `main()`:

```
1 System.out.println(p.getMobil().getMerk());
```

Kode 15: Tambahan pada MainPercobaan2.java

✓ **Checkpoint:** baris baru menampilkan `Avanza`.

## Pertanyaan Percobaan 2

1. Perhatikan class Pelanggan. Pada baris program manakah yang menunjukkan bahwa class Pelanggan memiliki relasi dengan class Mobil dan class Sopir?

- Perhatikan method `hitungBiayaSopir` pada class Sopir, serta method `hitungBiayaMobil` pada class Mobil. Mengapa method tersebut harus memiliki argument `hari`, padahal `hari` sendiri adalah atribut milik Pelanggan, bukan milik Mobil atau Sopir?
- Perhatikan kode dari class Pelanggan. Untuk apakah perintah `mobil.hitungBiayaMobil(hari)` dan `sopir.hitungBiayaSopir(hari)`?
- Perhatikan class MainPercobaan2. Untuk apakah sintaks `p.setMobil(m)` dan `p.setSopir(s)`?
- Untuk apakah proses `p.hitungBiayaTotal()`?
- Pada Langkah 7, `p.getMobil().getMerk()` memanggil dua method sekaligus secara berantai. Jelaskan urutan eksekusinya: objek apa yang dikembalikan `p.getMobil()`, dan objek apa yang kemudian dipanggil `.getMerk()`-nya?
- Andaikan `p.setMobil(m)` tidak pernah dipanggil lalu `p.hitungBiayaTotal()` dijalankan, error apa yang akan muncul? Jelaskan mengapa error itu terjadi, dikaitkan dengan konsep **referensi objek** yang sudah kita pelajari sebelumnya.

### Percobaan 3: Aggregation dengan Dua Role ke Kelas yang Sama (Kereta Api)

Sebuah **Kereta Api** dioperasikan oleh seorang **Masinis** dan seorang **Asisten Masinis**. Keduanya adalah **Pegawai PT**. Kereta Api Indonesia, artinya `masinis` dan `asisten` sama-sama bertipe **Pegawai**, hanya beda **peran (role)**.

**Info:** Ini **bukan** contoh multiplicity satu-ke-banyak (itu baru muncul di Percobaan 4). Di sini ada **dua relasi terpisah** ke class Pegawai yang sama, masing-masing dengan multiplicity 1, bukan satu relasi dengan multiplicity 2. Bedanya penting: kalau besok ada Kereta Api dengan 3 kondektur, itu baru butuh array (seperti Percobaan 4), bukan menambah atribut `asisten2`, `asisten3`, dst.

#### Langkah 1: Diagram kelas

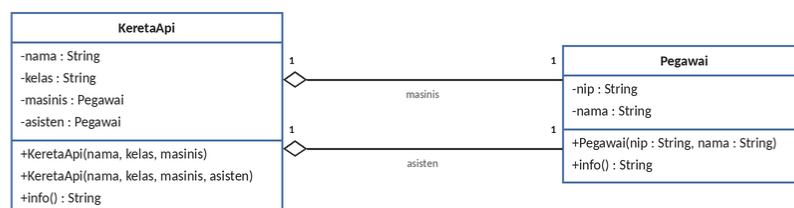


Figure 3: Dua garis relasi terpisah dari KeretaApi ke Pegawai, satu berperan sebagai “masinis”, satu lagi sebagai “asisten”. Keduanya multiplicity 1-1.

#### Langkah 2: Buat class Pegawai

```

1 package id.ac.polinema.relasticlass.percobaan3;
2
3 public class Pegawai {
4     private String nip;
5     private String nama;
6
7     public Pegawai(String nip, String nama) {
8         this.nip = nip;
9         this.nama = nama;
10    }
11
12    public void setNip(String nip) {
13        this.nip = nip;
14    }
15
16    public String getNip() {

```

```

17     return nip;
18 }
19
20 public void setNama(String nama) {
21     this.nama = nama;
22 }
23
24 public String getNama() {
25     return nama;
26 }
27
28 public String info() {
29     String info = "";
30     info += "Nip: " + this.nip + "\n";
31     info += "Nama: " + this.nama + "\n";
32     return info;
33 }
34 }

```

Kode 16: Pegawai.java lengkap. Perhatikan: class ini tidak punya constructor default, **nip** dan **nama** wajib diisi saat objek dibuat.

⚠ **Jika gagal:** karena Pegawai hanya punya constructor berparameter, `new Pegawai()` (tanpa argumen) akan gagal di-compile. Ini disengaja untuk memastikan setiap Pegawai selalu punya **nip** dan **nama** sejak awal dibuat.

### Langkah 3: Buat class KeretaApi

```

1 package id.ac.polinema.relasticlass.percobaan3;
2
3 public class KeretaApi {
4     private String nama;
5     private String kelas;
6     private Pegawai masinis;
7     private Pegawai asisten;
8 }

```

Kode 17: KeretaApi.java: dua atribut bertipe Pegawai, dibedakan namanya (masinis, asisten)

### Langkah 4: Tambah dua constructor KeretaApi

Satu constructor **tanpa** asisten (3 parameter), satu lagi **dengan** asisten (4 parameter). Percobaan ini akan menunjukkan apa yang terjadi kalau **asisten** tidak pernah diisi:

```

1 public KeretaApi(String nama, String kelas, Pegawai masinis) {
2     this.nama = nama;
3     this.kelas = kelas;
4     this.masinis = masinis;
5 }
6
7 public KeretaApi(String nama, String kelas, Pegawai masinis, Pegawai asisten) {
8     this.nama = nama;
9     this.kelas = kelas;
10    this.masinis = masinis;
11    this.asisten = asisten;
12 }

```

Kode 18: Tambahan pada KeretaApi.java: constructor 3 parameter dan 4 parameter

### Langkah 5: Tambah setter/getter dan info()

```

1 public void setMasinis(Pegawai masinis) {
2     this.masinis = masinis;
3 }
4
5 public Pegawai getMasinis() {

```

```

6         return masinis;
7     }
8
9     public void setAsisten(Pegawai asisten) {
10         this.asisten = asisten;
11     }
12
13     public Pegawai getAsisten() {
14         return asisten;
15     }
16
17     public String info() {
18         String info = "";
19         info += "Nama: " + this.nama + "\n";
20         info += "Kelas: " + this.kelas + "\n";
21         info += "Masinis: " + this.masinis.info() + "\n";
22         info += "Asisten: " + this.asisten.info() + "\n";
23         return info;
24     }

```

Kode 19: Tambahan pada KeretaApi.java: setter, getter, dan info()

## Langkah 6: Uji dengan kedua atribut terisi

```

1 package id.ac.polinema.relasiclass.percobaan3;
2
3 public class MainPercobaan3 {
4     public static void main(String[] args) {
5         Pegawai masinis = new Pegawai("1234", "Spongebob Squarepants");
6         Pegawai asisten = new Pegawai("4567", "Patrick Star");
7         KeretaApi keretaApi = new KeretaApi("Gaya Baru", "Bisnis", masinis, asisten);
8         System.out.println(keretaApi.info());
9     }
10 }

```

Kode 20: MainPercobaan3.java

☑ **Checkpoint:** output menampilkan info lengkap: Nama, Kelas, Masinis (Spongebob), dan Asisten (Patrick).

## Langkah 7: Kenali NullPointerException lebih dulu

Sebelum masuk ke tantangan berikutnya, pahami dulu polanya: **NullPointerException** selalu muncul ketika kita memanggil method pada sebuah referensi objek yang masih **null** (belum pernah di-**new**). Kalau **this.asisten** tidak pernah diisi, maka **this.asisten.info()** sama saja dengan memanggil method pada “kekosongan”, dan Java akan menghentikan program dengan error ini.

📌 **Catatan Konsep:** Pola pengamanannya disebut **guard clause**: cek dulu apakah referensinya **null** sebelum memanggil method-nya, misalnya **if (this.asisten != null) { ... }**. Simpan pola ini dan kita akan memakainya sebentar lagi.

## Langkah 8: Reproduksi bug: KeretaApi tanpa asisten

Buat class baru **MainPertanyaan** pada package yang sama:

```

1 package id.ac.polinema.relasiclass.percobaan3;
2
3 public class MainPertanyaan {
4     public static void main(String[] args) {
5         Pegawai masinis = new Pegawai("1234", "Spongebob Squarepants");
6         KeretaApi keretaApi = new KeretaApi("Gaya Baru", "Bisnis", masinis);

```

```

7      System.out.println(keretaApi.info());
8  }
9  }

```

Kode 21: MainPertanyaan.java: KeretaApi dibuat lewat constructor 3 parameter (tanpa asisten)

⚠ **Jika gagal:** program akan berhenti dengan `NullPointerException` pada baris `this.asisten.info()` di dalam `KeretaApi.info()`. Ini **disengaja**, lanjutkan ke langkah berikutnya untuk memperbaikinya.

### Langkah 9: Perbaiki dengan guard clause

Ubah method `info()` pada `KeretaApi.java` supaya baris **Asisten: ...** hanya dicetak kalau **asisten** benar-benar terisi:

```

1  public String info() {
2      String info = "";
3      info += "Nama: " + this.nama + "\n";
4      info += "Kelas: " + this.kelas + "\n";
5      info += "Masinis: " + this.masinis.info() + "\n";
6      if (this.asisten != null) {
7          info += "Asisten: " + this.asisten.info() + "\n";
8      }
9      return info;
10 }

```

Kode 22: KeretaApi.java setelah diperbaiki dengan guard clause

✅ **Checkpoint:** jalankan ulang MainPertanyaan, program kini selesai tanpa error, dan baris "Asisten: ..." tidak ikut tercetak.

### Pertanyaan Percobaan 3

1. Di dalam method `info()` pada class `KeretaApi`, baris `this.masinis.info()` dan `this.asisten.info()` digunakan untuk apa?
2. Apa hasil output dari MainPertanyaan **sebelum** diperbaiki (Langkah 8)? Mengapa hal tersebut dapat terjadi?
3. Kaitkan dengan materi referensi objek: apa isi variabel **asisten** di dalam objek `KeretaApi` yang dibuat lewat constructor 3-parameter, sebelum guard clause ditambahkan?
4. Setelah guard clause ditambahkan (Langkah 9), apakah objek **masinis** juga perlu dicek dengan cara yang sama? Perhatikan kedua constructor `KeretaApi`, apakah mungkin **masinis** bernilai **null**? Jelaskan.
5. Kelas Pegawai dipakai lewat dua atribut berbeda (**masinis** dan **asisten**) pada `KeretaApi`. Apakah ini membuat `KeretaApi` punya **dua objek Pegawai yang berbeda**, atau **satu objek Pegawai yang dipakai dua kali**? Jelaskan berdasarkan kode pada Langkah 6.

### Percobaan 4: Array of Object dan Multiplicity (Gerbong, Kursi, dan Penumpang)

Pada Percobaan 1-3, tiap relasi selalu satu-ke-satu. Tapi kadang satu objek **benar-benar** memiliki banyak objek lain sekaligus, dan jumlahnya bisa berubah, ini disebut **multiplicity**. Sebuah **Gerbong** kereta api punya banyak **Kursi**, dan tiap **Kursi** (kalau sudah terisi) punya satu **Penumpang**.

Studi kasus ini sekaligus jadi contoh penting: dua relasi has-a yang bertetangga (Gerbong-Kursi dan Kursi-Penumpang) ternyata **berbeda jenisnya** kalau diperiksa dengan kriteria kode, meskipun sekilas terlihat sama-sama “kumpulan objek”. Perhatikan baik-baik di Langkah 3 dan 4 nanti, siapa yang memanggil **new** untuk membuat objek Kursi, dan siapa yang memanggil **new** untuk membuat objek Penumpang.

Tabel notasi multiplicity berikut akan dipakai pada diagram di bawah:

| Multiplicity | Keterangan                                       |
|--------------|--------------------------------------------------|
| 0..1         | 0 atau 1 instance                                |
| 1            | Tepat 1 instance                                 |
| 0..*         | 0 atau lebih instance                            |
| 1..*         | Setidaknya 1 instance                            |
| n            | Tepat n instance (n diganti dengan sebuah angka) |
| m..n         | Setidaknya m instance, tetapi tidak lebih dari n |

## Langkah 1: Diagram kelas



Figure 4: Diamond ◇ terbuka di sisi Kursi (Kursi HAS-A 0..1 Penumpang, **Aggregation** (kursi boleh kosong, dan Penumpang-nya dibuat di luar Kursi) versus diamond ◆ terisi di sisi Gerbong (Gerbong HAS-A 1..\* Kursi, **Composition**, lihat Langkah 4: seluruh Kursi dibuat sendiri oleh Gerbong lewat **initKursi()**).

**Catatan Konsep:** Multiplicity 1..\* pada Gerbong-Kursi berbeda dari Percobaan 3: di sana ada 2 relasi terpisah bernama tetap (masinis, asisten). Di sini jumlah Kursi **tidak tetap** dan **tidak punya nama masing-masing**. Oleh karena itu dipakai **array**, bukan atribut bernama satu-satu. Array-nya sendiri **tidak menentukan** jenis relasinya (Aggregation atau Composition), itu tetap ditentukan oleh kriteria kode, seperti yang akan terlihat di Langkah 4.

## Langkah 2: Buat class Penumpang

```

1 package id.ac.polinema.relasiclass.percobaan4;
2
3 public class Penumpang {
4     private String ktp;
5     private String nama;
6
7     public Penumpang(String ktp, String nama) {
8         this.ktp = ktp;
9         this.nama = nama;
10    }
11
12    public String getKtp() {
13        return ktp;
14    }
15
16    public String getNama() {
17        return nama;
18    }
19
20    public String info() {

```

```

21     String info = "";
22     info += "Ktp: " + ktp + "\n";
23     info += "Nama: " + nama + "\n";
24     return info;
25 }
26 }

```

Kode 23: Penumpang.java lengkap

### Langkah 3: Buat class Kursi (has-a 0..1 Penumpang)

```

1  package id.ac.polinema.relasticlass.percobaan4;
2
3  public class Kursi {
4      private String nomor;
5      private Penumpang penumpang;
6
7      public Kursi(String nomor) {
8          this.nomor = nomor;
9      }
10
11     public void setPenumpang(Penumpang penumpang) {
12         this.penumpang = penumpang;
13     }
14
15     public Penumpang getPenumpang() {
16         return penumpang;
17     }
18
19     public String info() {
20         String info = "";
21         info += "Nomor: " + nomor + "\n";
22         if (this.penumpang != null) {
23             info += "Penumpang: " + penumpang.info() + "\n";
24         }
25         return info;
26     }
27 }

```

Kode 24: Kursi.java lengkap, perhatikan guard clause `if (this.penumpang != null)`, pola yang sama seperti yang kita terapkan sendiri di Percobaan 3

**Catatan Konsep:** Berbeda dari `KeretaApi.asisten` (Percobaan 3) yang boleh diisi belakangan tapi dianggap wajib ada, `Kursi.penumpang` memang wajar kosong. Kursi yang belum diduduki bukan bug, melainkan kondisi normal. Karena itu guard clause di sini bukan "perbaikan bug" seperti di Percobaan 3, melainkan bagian dari desain sejak awal.

### Langkah 4: Buat class Gerbong (has-a 1..\* Kursi, Composition)

```

1  package id.ac.polinema.relasticlass.percobaan4;
2
3  public class Gerbong {
4      private String kode;
5      private Kursi[] arrayKursi;
6
7      public Gerbong(String kode, int jumlah) {
8          this.kode = kode;
9          this.arrayKursi = new Kursi[jumlah];
10         this.initKursi();
11     }
12
13     private void initKursi() {
14         for (int i = 0; i < arrayKursi.length; i++) {
15             this.arrayKursi[i] = new Kursi(String.valueOf(i + 1));
16         }
17     }
18 }

```

Kode 25: Gerbong.java: constructor langsung mengisi seluruh array Kursi lewat `initKursi()`

■ **Catatan Konsep:** `initKursi()` bersifat **private** dan langsung dipanggil di dalam constructor `Gerbong` sendiri. Pemanggil dari luar (`MainPercobaan4`) tidak pernah membuat objek `Kursi` sendiri. Semua `Kursi` otomatis “siap pakai” begitu `Gerbong` selesai dibuat. Terapkan kriteria kode dari Capaian Praktikum: karena `Gerbong` sendiri yang memanggil `new Kursi(...)` (tidak ada parameter bertipe `Kursi` ataupun `Kursi[]` pada constructor-nya), relasi `Gerbong-Kursi` ini sebenarnya **Composition**, bukan **Aggregation**. Pola yang sama akan kita pelajari lebih formal di Percobaan 5, hanya saja di sana part-nya cuma satu objek, sedangkan di sini part-nya berupa array (banyak objek `Kursi` sekaligus).

### Langkah 5: Tambah method navigasi ke Kursi tertentu

```
1 public void setPenumpang(Penumpang penumpang, int nomor) {
2     this.arrayKursi[nomor - 1].setPenumpang(penumpang);
3 }
4
5 public String info() {
6     String info = "";
7     info += "Kode: " + kode + "\n";
8     for (Kursi kursi : arrayKursi) {
9         info += kursi.info();
10    }
11    return info;
12 }
```

Kode 26: Tambahan pada `Gerbong.java`

### Langkah 6: Uji di `MainPercobaan4`

```
1 package id.ac.polinema.relasticlass.percobaan4;
2
3 public class MainPercobaan4 {
4     public static void main(String[] args) {
5         Penumpang p = new Penumpang("12345", "Mr. Krab");
6         Gerbong gerbong = new Gerbong("A", 10);
7         gerbong.setPenumpang(p, 1);
8         System.out.println(gerbong.info());
9     }
10 }
```

Kode 27: `MainPercobaan4.java`

✓ **Checkpoint:** output mencetak 10 baris `Kursi` (Nomor: 1 sampai 10); hanya `Kursi` nomor 1 yang menampilkan baris “Penumpang: ...” (Mr. Krab).

⚠ **Jika gagal:** `ArrayIndexOutOfBoundsException` artinya kita mengisi indeks yang tidak ada, misalnya `gerbong.setPenumpang(p, 11)` padahal array-nya cuma berukuran 10 (indeks `kursi` yang valid: nomor 1 sampai 10, karena `nomor - 1` dipakai sebagai indeks array yang dimulai dari 0).

### Pertanyaan Percobaan 4

1. Pada main program dalam class `MainPercobaan4`, berapakah jumlah `kursi` dalam `Gerbong A`?
2. Perhatikan potongan kode `if (this.penumpang != null) { ... }` pada method `info()` dalam class `Kursi`. Apa maksud kode tersebut?
3. Mengapa pada method `setPenumpang()` dalam class `Gerbong`, nilai `nomor` dikurangi dengan angka 1?

4. Instansiasi objek baru **budi** dengan tipe Penumpang, kemudian masukkan objek baru tersebut pada gerbong dengan `gerbong.setPenumpang(budi, 1)`, menimpa Mr. Krab yang sudah duduk di sana. Apakah yang terjadi? Apakah Java memberi peringatan/error?
5. Modifikasi program sehingga **tidak diperkenankan** menduduki kursi yang sudah ada penumpang lain (tambahkan pengecekan pada `Gerbong.setPenumpang()` sebelum baris `arrayKursi[nomor - 1].setPenumpang(...)` dijalankan).
6. Bandingkan tiga bentuk relasi has-a yang sudah kita praktikkan: Laptop-Processor (Percobaan 1, 1-1), KeretaApi-Pegawai (Percobaan 3, dua relasi 1-1 bernama), dan Gerbong-Kursi (Percobaan 4, 1..\*). Untuk kasus seperti apa kita akan memilih **array**, dan untuk kasus seperti apa kita akan memilih **atribut bernama satu-satu**?
7. Terapkan **kriteria kode** (siapa yang memanggil **new**) pada dua relasi has-a di Percobaan ini: Gerbong-Kursi dan Kursi-Penumpang. Manakah yang Aggregation dan manakah yang Composition? Tunjukkan baris kode yang menjadi bukti untuk masing-masing.

## Percobaan 5: Composition (Mobil dan Mesin)

Coba ingat lagi Percobaan 4: method `initKursi()` pada Gerbong memanggil `new Kursi()` sendiri, tanpa pernah menerima objek Kursi dari luar. Berdasarkan kriteria kode, itu sebenarnya sudah **Composition**, hanya saja belum dinamai secara eksplisit di sana. Percobaan 1-3 murni **Aggregation**: objek “part”-nya selalu dibuat **di luar** class whole, lalu di-inject lewat constructor atau setter. Sekarang giliranmu mempraktikkan pola Composition itu secara sadar, dengan studi kasus baru: sebuah **Mobil** yang membuat **Mesin**-nya **sendiri**, tanpa pernah menerimanya dari luar.

### Langkah 1: Diagram kelas

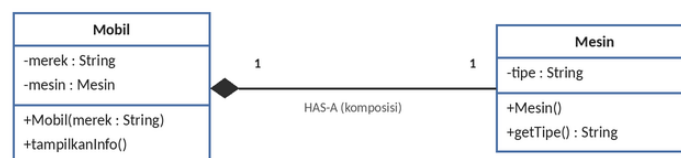


Figure 5: Diamond ◆ (terisi penuh) = Composition. Perhatikan: class Mobil **tidak** punya `setMesin()`, ini disengaja, bukan lupa ditulis.

**Catatan Konsep:** Diamond terisi ◆ menandakan **kepemilikan eksklusif dan lifecycle yang terikat**: kalau objek Mobil dihapus/hilang, objek Mesin miliknya ikut hilang, karena tidak ada pihak lain yang pernah memegang referensi ke Mesin itu selain Mobil itu sendiri.

### Langkah 2: Buat class Mesin

```

1 package id.ac.polinema.relasticlass.percobaan5;
2
3 public class Mesin {
4     private String tipe;
5
6     public Mesin() {
7         this.tipe = "4-silinder";
8     }
9
10    public String getTipe() {
11        return tipe;
12    }
13 }
  
```

Kode 28: Mesin.java: constructor default langsung menetapkan nilai tipe

 **Info:** Mesin sengaja dibuat sesederhana mungkin (tanpa constructor berparameter, tanpa setter) supaya perhatian kita fokus ke class Mobil di langkah berikutnya.

### Langkah 3: Buat class Mobil — Mesin dibuat sendiri di dalam constructor

```
1 package id.ac.polinema.relasticlass.percobaan5;
2
3 public class Mobil {
4     private String merek;
5     private Mesin mesin;
6
7     public Mobil(String merek) {
8         this.merek = merek;
9         this.mesin = new Mesin();
10    }
11 }
```

*Kode 29: Mobil.java: constructor Mobil sendiri yang memanggil new Mesin()*

 **Catatan Konsep:** Bandingkan dengan `Laptop(String merk, Processor proc)` pada Percobaan 1: di sana `Processor` diterima sebagai parameter. Di sini, constructor `Mobil` hanya menerima `merek` dan tidak ada parameter bertipe `Mesin` sama sekali. Mesin dibuat sendiri oleh `Mobil`, memakai `new Mesin()` di baris kode milik class `Mobil` sendiri. Inilah kriteria kode pembeda Aggregation vs Composition.

### Langkah 4: Tambah `tampilkanInfo()`, sengaja tanpa getter/setter Mesin

```
1     public void tampilkanInfo() {
2         System.out.println("Mobil: " + merek);
3         System.out.println("Mesin: " + mesin.getTipe());
4     }
```

*Kode 30: Tambahan pada Mobil.java (sisipkan sebelum kurung kurawal penutup class)*

 **Jika gagal:** jangan menambahkan `setMesin(Mesin mesin)` atau `getMesin()`: `Mesin` pada class ini, meski NetBeans mungkin menyarakannya lewat auto-generate getter/setter. Tanpa setter, tidak ada cara bagi kode di luar `Mobil` untuk mengganti `Mesin` yang sudah dibuat. Itulah yang menjaga relasi ini tetap Composition (lihat Pertanyaan no. 2 di bawah).

### Langkah 5: Uji di `MainPercobaan5`

```
1 package id.ac.polinema.relasticlass.percobaan5;
2
3 public class MainPercobaan5 {
4     public static void main(String[] args) {
5         Mobil mobil = new Mobil("Avanza");
6         mobil.tampilkanInfo();
7     }
8 }
```

*Kode 31: MainPercobaan5.java*

 **Checkpoint:** output menampilkan `Mobil: Avanza` lalu `Mesin: 4-silinder`. Walau kita tidak pernah menulis `new Mesin()` di `MainPercobaan5` sama sekali.

## Pertanyaan Percobaan 5

1. Pada class Mobil, baris manakah yang menunjukkan bahwa Mesin adalah bagian yang “dimiliki secara eksklusif” oleh Mobil (bukan sekadar “dipinjam”)?
2. Apa yang terjadi secara desain jika ditambahkan method `setMesin(Mesin mesin)` pada class Mobil? Apakah relasi ini akan tetap menjadi Composition? Jelaskan.
3. Bandingkan dengan Percobaan 1 (Laptop-Processor): sebutkan satu perbedaan **baris kode** yang membuat salah satunya Aggregation dan yang lain Composition.
4. Jika objek `mobil` di `MainPercobaan5` di-set `null` setelah `tampilkanInfo()` dipanggil, apa yang terjadi pada objek Mesin miliknya? Bandingkan dengan nasib objek Processor pada Percobaan 1 seandainya objek Laptop-nya dihapus, apakah Processor tersebut masih bisa “diselamatkan” oleh kode lain? Kenapa Mesin tidak bisa?
5. Coba (secara terpisah, boleh di file/package percobaan sendiri) tambahkan constructor kedua pada Mobil yang menerima parameter `Mesin`, mirip pola Percobaan 1: `public Mobil(String merek, Mesin mesin) { this.merek = merek; this.mesin = mesin; }`. Kalau constructor ini yang dipakai, apakah Mobil-Mesin berubah menjadi Aggregation? Jelaskan alasannya.

## Percobaan 6: Dependency / Uses-A (Laptop Mencetak Dokumen ke Printer)

Relasi terakhir yang perlu kita bedakan dari Aggregation dan Composition adalah **Dependency** (uses-a): satu class memakai class lain **hanya sesaat**, lewat parameter sebuah method, tanpa pernah menyimpannya sebagai atribut.

### Langkah 1: Diagram kelas



Figure 6: Garis putus-putus dengan panah terbuka (bukan diamond!) menandakan Dependency: Laptop “memakai sesaat” objek Printer, tanpa menyimpannya.

**Catatan Konsep:** Perhatikan: **tidak ada atribut Printer pada class Laptop**. Ini beda mendasar dari Aggregation maupun Composition, yang keduanya **menyimpan** objek part sebagai atribut. Dependency hanya “meminjam” objek selama satu pemanggilan method saja.

### Langkah 2: Buat class Printer

```
1 package id.ac.polinema.relasiclass.percobaan6;
2
3 public class Printer {
4     private String merk;
5
6     public Printer(String merk) {
7         this.merk = merk;
8     }
9
10    public void cetak(String namaFile) {
11        System.out.println("[ " + merk + " ] Mencetak " + namaFile + "...");
12        System.out.println("[ " + merk + " ] Selesai.");
13    }
14 }
```

Kode 32: Printer.java lengkap

### Langkah 3: Buat class Laptop dengan method cetakDokumen()

Kali ini Laptop dibuat sederhana (hanya atribut **merk**), lalu ditambah satu method yang **menerima** Printer sebagai parameter:

```
1 package id.ac.polinema.relasticlass.percobaan6;
2
3 public class Laptop {
4     private String merk;
5
6     public Laptop(String merk) {
7         this.merk = merk;
8     }
9
10    public void cetakDokumen(Printer printer, String namaFile) {
11        System.out.println(merk + " mengirim dokumen ke printer...");
12        printer.cetak(namaFile);
13    }
14 }
```

*Kode 33: Laptop.java: Printer hanya muncul sebagai parameter method, tidak pernah menjadi atribut*

**Catatan Konsep:** Setelah **cetakDokumen()** selesai dijalankan, Laptop **tidak lagi “mengenal”** objek **printer** yang tadi dipakainya. Parameter method hanya hidup selama method itu berjalan. Kalau kita ingin Laptop tetap mengingat printer yang sama untuk dipakai berkali-kali, kita harus menyimpannya sebagai atribut (lihat Pertanyaan no. 4). Dan begitu itu terjadi, relasinya bukan Dependency lagi.

### Langkah 4: Uji di MainPercobaan6

```
1 package id.ac.polinema.relasticlass.percobaan6;
2
3 public class MainPercobaan6 {
4     public static void main(String[] args) {
5         Laptop laptop = new Laptop("Thinkpad");
6         Printer printer = new Printer("Epson L3110");
7         laptop.cetakDokumen(printer, "Laporan.pdf");
8     }
9 }
```

*Kode 34: MainPercobaan6.java*

**Checkpoint:** output menampilkan tiga baris: “Thinkpad mengirim dokumen...”, “[Epson L3110] Mencetak Laporan.pdf...”, “[Epson L3110] Selesai.”.

### Pertanyaan Percobaan 6

1. Apakah class Laptop pada percobaan ini memiliki atribut bertipe Printer? Bandingkan dengan Percobaan 1, di mana Processor disimpan sebagai atribut Laptop.
2. Setelah method **cetakDokumen()** selesai dijalankan, apakah Laptop masih menyimpan referensi ke objek **printer** yang tadi dipakai? Jelaskan berdasarkan baris kode class Laptop.
3. Mengapa relasi Laptop-Printer pada percobaan ini disebut Dependency (uses-a), bukan Aggregation, meskipun sama-sama melibatkan dua objek yang saling berinteraksi?
4. Coba ubah kode Laptop supaya Printer disimpan sebagai atribut (mis. **private Printer printerDefault**, diisi lewat constructor atau setter, lalu dipakai kembali di **cetakDokumen()** tanpa parameter Printer). Apakah relasi ini sekarang berubah dari Dependency menjadi Aggregation? Jelaskan.

- Lengkapi tabel berikut dengan kata-katamu sendiri (boleh dijawab di laporan): untuk masing-masing dari Aggregation, Composition, dan Dependency, sebutkan (a) apakah objek part disimpan sebagai atribut atau tidak, dan (b) siapa yang memanggil **new** untuk membuat objek part tersebut.

## D. Tugas dan Deliverable

Kumpulkan hal berikut sesuai format yang diminta Dosen:

- Screenshot output program untuk **setiap** Percobaan (1 sampai 6).
- Source code seluruh package **relasiclass.percobaan1** s.d. **percobaan6**.

### Tugas mandiri:

- Rancang satu studi kasus sendiri (bebas topiknya, misalnya perpustakaan, klinik, toko online, dsb.), gambarkan diagram kelasnya, lalu implementasikan ke dalam program. Studi kasus **wajib** melibatkan minimal 4 class (class yang berisi **main** tidak dihitung) dan **wajib mencakup ketiganya**: minimal satu relasi **Aggregation**, satu **Composition**, dan satu **Dependency**. Tandai pada laporanmu, bagian mana dari kode yang merupakan masing-masing jenis relasi tersebut, dan sertakan alasannya.
- Jawab singkat (3-5 kalimat): dalam merancang sistem barumu sendiri, bagaimana kita memutuskan sebuah relasi antar class seharusnya Aggregation, Composition, atau Dependency? Sebutkan pertanyaan kunci yang kita ajukan ke diri sendiri saat memutuskan.

 **Info:** Tugas mandiri sengaja mewajibkan ketiga jenis relasi (bukan hanya Aggregation seperti pada versi jobsheet sebelumnya), supaya kita benar-benar mempraktikkan seluruh spektrum relasi has-a yang sudah dibahas di kelas, bukan cuma satu jenis saja.

## E. Kriteria Penilaian

| Komponen                           | Bobot | Kriteria Lengkap (100%)                                                                                                      | Kriteria Minimum                                                                          |
|------------------------------------|-------|------------------------------------------------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------------|
| Langkah kerja Percobaan 1-6 tuntas | 35%   | Seluruh langkah dijalankan dan berfungsi sesuai checkpoint                                                                   | Sebagian besar langkah selesai, hasil akhir tiap Percobaan berjalan                       |
| Checkpoint terverifikasi           | 20%   | Semua checkpoint tercapai dan dibuktikan (screenshot/output)                                                                 | Sebagian checkpoint terbukti                                                              |
| Pertanyaan pemahaman terjawab      | 20%   | Seluruh pertanyaan (termasuk yang bertanda “baru”) terjawab dengan alasan yang tepat dan merujuk baris kode yang relevan     | Sebagian besar pertanyaan terjawab, sebagian alasan kurang tepat                          |
| Tugas mandiri                      | 25%   | Studi kasus benar, minimal 4 class, ketiga jenis relasi (Aggregation, Composition, Dependency) ada dan ditandai dengan tepat | Studi kasus ada meski belum mencakup ketiga jenis relasi atau penjelasannya belum lengkap |